

AM5 Server Plugin and Module Handler Developer Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - AM5 Server Plugin and Module Handler Developer Guide.....	3
2. Introduction to AM5 Server Plugin and Module Handler Development.....	4
3. Server Plugin and Module.....	7
3.1. Plugin.....	8
3.2. Module Handler.....	12
3.3. Custom Response Collector.....	25
3.4. Non-Blocking Event Listener.....	34
3.5. Plugin Service.....	39
3.6. JMS Integration (Message Driven Bean).....	47
3.7. User Auto-Provisioning for SAML Service Provider.....	52

1. Preface - AM5 Server Plugin and Module Handler Developer Guide

AccessMatrix (AM5) supports the use of a Java server plugin or module handler to extend its functionalities. This document provides information on how to do so.

Intended Reader

This guide is intended as a reference for product architects and developers (i.e. business partners, clients, or Professional Services staff).

Document Scope

This document provides information on how to write a Java server plugin and module handler for AccessMatrix 5.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix XML-RPC and SOAP API Reference*
- [AccessMatrix REST API Specification](#)

2. Introduction to AM5 Server Plugin and Module Handler Development

AccessMatrix server functionalities can be extended by writing Java server plugin or module handler through supported interfaces. The developer can create a custom module handler, custom event listener, custom delivery channel handler, server-side plugin, and login module handler, all of which will be discussed in this document.

Terms/Acronyms and Definitions

The following table presents some of the terms and acronyms used in this document:

Term/Acronym	Definition
UAM	Universal Access Management
WSA	Web Security Agent
AM	Access Manager
AccessMatrix	Refer to i-Sprint suite of IAM products
SOAP	Simple Object Access Protocol
XML-RPC	XML for Remote Procedure Call
JNI	Java Native Interface
PS	Professional Services
a.k.a.	also known as
<ACMATRIX_HOME>	AccessMatrix Home Directory, e.g. "C:\Program Files\AccessMatrix\tomcat"

Type of Server Plugin/Module

The following custom plugins/modules are supported:

1. Server-side plugin
 2. Login module handler (a.k.a. Pluggable Authentication Module, or PAM)
 3. Custom response collector
 4. Custom event listener
 5. Custom delivery channel handler
-

Each type of server module will be discussed in its own section.

Prerequisites

JDK Version

Currently, AccessMatrix 5 server is supported on JDK 1.6, 1.7, and 1.8. Developers must first identify the target JDK version so that an appropriate compiler is used.

Required libraries

The main interfaces to be implemented are in **am.jar** and **am-consts.jar**, within the *WEB-INF/lib* directory of the deployed AM5 web application. Optionally, log4j, available at *WEB-INF/lib/oss-org-apache-log4j-x.x.x.jar*, is required for compilation for logging purposes.

JNDI Data Source/JMS

Normally, custom code written must access an external persistence storage or queue provider such as an RDBMS or MQ through standard JNDI data source or JMS interface. It is the responsibility of the developer to acquire the relevant client-side drivers / SDK jar files and test it before integrating this into AccessMatrix server.

Use of Open Source Components

Developers must ensure that:

1. The open source library or any dependent library used is properly licensed.
2. The libraries used are not in conflict with the list of libraries used in the AccessMatrix server.
Developers can check the list of jar files in the *WEB-INF/lib* directory of the deployed AccessMatrix web application.

SOAP/XML-RPC/JNI

If developers use a web service client (such as SOAP or XML-RPC) to talk to an external system within the server plugin/module, it may result in a conflict with the AccessMatrix server's own SOAP/XML-RPC libraries. Therefore, such an integration approach is strongly discouraged. If JNI is used, extreme caution must be exercised as the native component loaded through JNI may crash the AccessMatrix server.

Deployment

Each type of module has its own deployment model. Each will be discussed under the respective module's page.

Stress Test

It is the responsibility of the developers to ensure their plugin or module is properly stress tested for scalability and concurrent issues before the actual code is deployed into the AccessMatrix server. i-Sprint Innovations does not accept any responsibility for performance degradation or production problems arising from any such modules (unless they are developed by our Professional Services team).

SSL Connection

If the plugin or module requires JSSE connection to an external system, developers must first study how SSL is supported on the target web application server where the AccessMatrix server is deployed. This is because most web applications servers have a proprietary way of supporting an outbound JSSE/SSL connection.

3. Server Plugin and Module

The following pages provide information and samples for their respective modules:

- [Plugin](#)
- [Module Handler](#)
- [Custom Response Collector](#)
- [Non-Blocking Event Listener](#)
- [Plugin Service](#)
- [JMS Integration \(Message Driven Bean\)](#)

3.1. Plugin

The AM5 server-side plugin allows for the loading of a custom module handler, event listener, or the execution of other custom codes.

Interface

Java

```
/**
 * Interface representing a server plugin.
 * <p>Must be implemented by all server plugins. The
 * implementation class must provide a default
 * constructor. The default constructor only construct the
 * plugin object but should not initialize or
 * <code>load</code> the plugin. At the right timing
 * <code>Server</code> invokes <code>load</code> method.
 * <p>Initialization routine should be done in method
 * {@link #load}, and cleanup routine should be done in
 * {@link #unload}
 */
public interface IPlugin {
    /**
     * Returns the plugin's id
     */
    public String getId() ;
    public String getVersion() ;
    /**
     * Called when a <code>Server</code> loads this
     * plugin. To unload the plugin <code>Server</code> calls
     * unload method.
     * <p> All plugin class must have a default
     * constructor.
     * The default constructor by definition takes no
     * input parameters, constructs the plugin instance, but
     * should not perform initialization yet. The
     * initialiation code that modifies the plugin state and
     * variables should be done in load.
     * <p> Plugin should allow the load method after it
     * is unloaded. This happens when <code>Server</code> is
     * requested to reload the plugin and when executing
     * plugin auto-refresh.
     */
    public void load(IPluginContext pc) ;
    /**
     * Called when a <code>Server</code> unloads this
     * plugin. The plugin had previously been loaded by
     * <code>Server</code> by calling load method.
     * <p> Plugin should allow the load method after it
     * is unloaded. This happens when <code>Server</code> is
     * requested to reload the plugin and when executing
     * plugin auto-refresh.
     */
}
```

```
    public void unload(IPluginContext pc) ;  
}
```

Example

Java

```
package test;  
  
import com.isprint.am.core.event.IDeliveryChannelHandler;  
import com.isprint.am.dto.bean.EnumConstant.ObjectClass;  
import com.isprint.am.server.IPlugin;  
import com.isprint.am.server.IPluginContext;  
import test.delivery.SystemOutDeliverer;  
  
public class SystemOutDelivererPlugin implements IPlugin {  
    public SystemOutDelivererPlugin() {}  
  
    public String getId() {  
        return SystemOutDelivererPlugin.class.getSimpleName();  
    }  
  
    public String getVersion() {  
        return "1.0";  
    }  
  
    public void load(IPluginContext pc) {  
        // register a custom delivery channel  
  
        pc.getServerContext().addHandlerClass(SystemOutDeliverer.class,  
        ObjectClass.DeliveryChannel, IDeliveryChannelHandler.class);  
    }  
  
    public void unload(IPluginContext pc) {  
  
        pc.getServerContext().removeHandlerClass(SystemOutDeliverer.class,  
        ObjectClass.DeliveryChannel);  
    }  
}
```

Loading Custom Handler/Listener Examples

Java

```
// Loading custom delivery channel handler  
pc.getServerContext().addHandlerClass(SystemOutDeliverer.class,  
ObjectClass.DeliveryChannel, IDeliveryChannelHandler.class);  
// Loading custom login handler  
pc.getServerContext().addLoginModuleHandlerClass(CustomLoginHandler.class  
);  
// Loading custom lookup handler
```

```
pc.getServerContext().addLookupModuleHandlerClass(CustomLookupHandler.class);
// Register an event listener
pc.getServerContext().getEventService().addEventListener(new
EventCategoryTypePair[]{
    EventCategoryTypePair.valueOf(EventCategoryTO.Authentication,
EventTypeTO.Login),
    EventCategoryTypePair.valueOf(EventCategoryTO.Authentication,
EventTypeTO.Logout)
}, listener);
```

Reading plugin properties

The AM5 plugin framework automatically reads the plugin properties file based on the plugin Id (returned by the getId() method). The property inside the file must also be prefixed by plugin.<pluginId>. (e.g. plugin.SystemOutDelivererPlugin.someOtherPluginProperty)

Java

```
package test;

public String getId() {
    return
SystemOutDelivererPlugin.class.getSimpleName();
}

public void load(IPluginContext pc) {
    // read from plugin properties
    String value =
pc.getProperties().get("someOtherPluginProperty");
}
```

Deployment

The custom class must be packed into a jar file and placed in the <ACMATRIX_HOME>/plugins/ directory, where <ACMATRIX_HOME> is the app server's current directory. The plugin directory is also configurable in **amsystem.properties** using the following property: am.pluginidir

Any other dependent libraries should be put in WEB-INF/lib/ of the AM5 web app.

A server restart is required to load the plugin.

Plugin Properties File

The plugin property file is optional. The AccessMatrix Server uses it to enable/disable or control the loading order of the plugin. The plugin may also use it to configure itself. The properties file must have the same name as what is returned by the `getId()` method in the `IPlugin` interface. If your plugin Id is "SystemOutDelivererPlugin", then the property filename is **SystemOutDelivererPlugin.properties**. Possible content of the property file:

Properties (.properties files)

```
# the following must match the value returned by getId() in
# IPlugin interface
plugin.SystemOutDelivererPlugin.enabled=true
# load order
plugin.SystemOutDelivererPlugin.loadOnStartup=3

# Custom/additional property for the plugin
plugin.SystemOutDelivererPlugin.someOtherPluginProperty=testing
```

Logging

When plugin is being loaded, **am.log** should print out information regarding the plugin. For example:

Loading plugin instance, spId=..., version=..., instance=..., loadOrder=...

3.2. Module Handler

This section contains information on customizing module handlers.

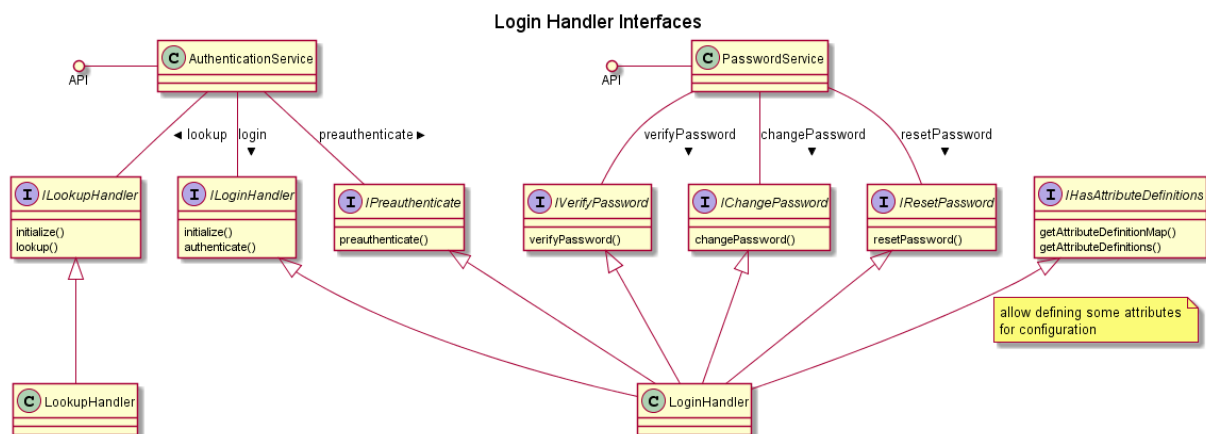
Login Module Handler

Authentication

The developer is assumed to be familiar with the concept of authentication realms. They should also know how to call the AM5 web service API and configure the authentication realm with the appropriate lookup and login module.

The login module handler is known as PAM (Pluggable Authentication Module). It is often required if the default list of login module handlers supplied by the AccessMatrix server cannot fulfil the authentication requirements of the client. For example, the customer may have a proprietary authentication server at the backend. In order to authenticate the user, MQ integration via a proprietary data format is required. In this case, a new login module handler must be written.

Interfaces



ILoginHandler (ILoginH)

This interface is required to support the XML-RPC / SOAP's `AuthenticationService.login` and `loginEx` methods.

com.isprint.am.core.auth.ILoginHandler

com.isprint.am.core.auth.ILoginHandler	
Java	<pre> /** * Initialize the module handler with server context and the * associated login module in a thread-safe manner * @param ctx the server context * @param baseObj the login module associated with the * handler */ public void initialize(IServerContext ctx, IBaseObject baseObj); /** * Authenticate the user with a session for a login module * within an authentication realm. If IVerifyPassword interface * is also implemented, * this method should delegate implementation to * IVerifyPassword.verifyPassword method. * * @param sessionContext the current session context where * getUser() method will return the user object * @param credential the credential for verification * @throws AuthenticationException.WrongPassword if * authentication failed due to wrong password (optional * challenge can be passed for challenge response * authentication) * @throws AuthenticationException.ChallengeResponse if a * challenge is issued and a response is required * @throws AuthenticationException.DeniedByPolicy if * authentication is denied due to violation of security policy * @throws AuthenticationException.OtherReasons if * authentication failed due to other reason */ public void authenticate(SessionContext sessionContext, ICredential credential, Map<String, String> params) throws AuthenticationException; </pre>

IVerifyPassword (optional)

If the PasswordService.verifyPassword interface is required at the client-side, the following interface must be implemented:

com.isprint.am.core.IModuleFeature.IVerifyPassword	
Java	<pre> /** * Verify credential of a user with no session generated * (used by PasswordService.verifyPassword) </pre>

com.isprint.am.core.IModuleFeature.IVerifyPassword

```
* @param user the user object
* @param credential the credential to be verified
*/
public interface IVerifyPassword extends IModuleFeature {
    void verifyPassword(ISession userSession, IUser user,
        ICredential credential, Map<String,String> params) throws
        AuthenticationException;
}
```

IResetPassword (optional)

If the PasswordService.resetPassword interface is required at the client-side, the following interface must be implemented:

com.isprint.am.core.IModuleFeature.IResetPassword

Java

```
/**
 * ModuleFeature interface representing
 <code>resetPassword</code> method of the
 <code>ILoginModule</code>
 */
public interface IResetPassword extends IModuleFeature {
    /**
     * Reset a user's password.
     * @param user target user
     * @param new password
     * @throws AuthenticationException if reset password
     failed
     */
    public void resetPassword(IEditableUser user, Password
        newPassword, Map<String, String> params) throws
        AuthenticationException;
}
```

IChangePassword (optional)

If the PasswordService.changePassword interface is required at the client-side, the following interface must be implemented:

com.isprint.am.core.IModuleFeature.IChangePassword

Java

com.isprint.am.core.IModuleFeature.IChangePassword

```
/**
 * ModuleFeature interface representing
 <code>changePassword</code> method of the
 <code>ILoginModule</code>
 */
public interface IChangePassword extends IModuleFeature {
    /**
     * Change user's own password.
     * @param user user object
     * @param oldPassword old password
     * @param newPassword new password
     * @throws AuthenticationException.VerificationFailed if
     old password is incorrect
     * @throws AuthenticationException.DeniedByPolicy if
     violation of password policy
     * @throws QueryException if there is an exception
     querying the backend persistence storage
     * @throws UpdateException if there is an exception
     updating the backend persistence storage
     */
    public void changePassword(ISession userSession, IUser
    user, Password oldPassword, Password newPassword,
    Map<String, String> params) throws AuthenticationException;
}
```

IPreauthenticate (optional)

Preauthentication is normally used to get some parameters from the login module handler to prepare the authentication process itself. Normally, it is used for E2EE (End-to-End Encryption).

If the AuthenticationService.preauthenticate interface is required at the client-side, the following interface must be implemented:

com.isprint.am.core.IModuleFeature.IPreauthenticate

Java

```
/**
 * ModuleFeature interface representing
 <code>preauthenticate</code> method
 */
public interface IPreauthenticate extends IModuleFeature {
    /**
     * Execute preauthentication to get parameters required by
     the authentication
     * @param params parameters required by the login module
     handler, if any
     * @throws AuthenticationException
     */
}
```

com.isprint.am.core.IModuleFeature.IPreauthenticate

```
*/  
ChallengeTO preauthenticate(Map<String,String> params)  
throws AuthenticationException;  
}
```

Initialization

All thread-safe initialization should be done inside the initialize method. The initialize method will be called when the login module handler is first used. Developers should log the initialization information under INFO or ERROR logging in **am.log**.

In the initialize() method, the login module object is provided, which may contain some attributes/configurations for the login module handler instance.

Logging

Developers should use the log4 logger used by the AccessMatrix server. By default, the log will go to **am.log**. However, a different logger can be configured via *WEB-INF/classes/amlog4j.properties* if necessary:

To use the log4j logger, declare a private static variable in your implementation class with the class name used in the getLogger parameter:

Java

```
private static final Logger log =  
    LogManager.getLogger(MyLoginModuleHandler.class);
```

Exception/Error Handling

A custom error code can be propagated back to the client by setting an extended error code and extended message into any com.isprint.am.dto.exception.AuthenticationException.

Java

```
public void setExtendedCode(int extendedCode);  
public void setExtendedMessage(String extendedMessage)
```

Developers are expected to select the correct AuthenticationException type from the one of the subclasses of AuthenticationException. For example:

AuthenticationException type	Used for
WrongPassword	If password is confirmed to be incorrect
ChallengeResponse	If a challenge is issued and a response is required
DeniedByPolicy, DeniedByPolicyOtpReuseNotAllowed, DeniedByPolicyTokenExpired	Denied by policy e.g. time/location restriction, otp reuse not allowed, token expired
OtherReasons	If you cannot find a more appropriate Exception, use this and set the extended code / extended message

Defined Configurable Attributes Managed in Admin Console

It is possible to use the Admin Console to manage the properties or attributes required by your custom module handler. This is done with the server-side Attribute Definition framework by implementing the interface IModuleFeature.IHasAttributeDefinitions with the getAttributeDefinitions() and getAttributeDefinitionMap() methods using a declarative style of attribute definitions. An example is given below for the RadiusLogin login module handler:

Java

```
public class RadiusLogin implements
ILoginHandler, IModuleFeature.IVerifyPassword, IModuleFeature.IHasAttribute
Definitions {
    static Logger logger = Logger.getLogger(RadiusLogin.class);
    public static String ATTR_RadiusSharedSecret = "RadiusSharedSecret";
    public static AttributeDefinition DEF_RadiusSharedSecret =
        new AttributeDefinition(ATTR_RadiusSharedSecret,
        DataType.STRING, EncryptionFlag.YES, true)

    public static String ATTR_RadiusServerIP = "RadiusServerIP";
    public static AttributeDefinition DEF_RadiusServerIP =
        new AttributeDefinition(ATTR_RadiusServerIP, DataType.STRING,
        EncryptionFlag.NO, true)
        .setDefaultValue("127.0.0.1");

    public static String ATTR_RadiusServerPort = "RadiusServerPort";
    public static AttributeDefinition DEF_RadiusServerPort =
        new AttributeDefinition(ATTR_RadiusServerPort,
        DataType.INTEGER, EncryptionFlag.NO, true)
```

```

        .setDefaultValue(1812).setFloor(1).setCeiling(65535);

    public static final IAttributeDefinition[] ATTRIBUTE_DEFINITIONS =
new IAttributeDefinition[] {
    DEF_RadiusSharedSecret,
    DEF_RadiusServerIP,
    DEF_RadiusServerPort
};

    public final static Map<String, IAttributeDefinition>
ATTRIBUTE_DEFINITIONS_MAP =
AttributeDefinition.arrayAsDefinitionMap(ATTRIBUTE_DEFINITIONS);

    public IAttributeDefinition[] getAttributeDefinitions() {
        return ATTRIBUTE_DEFINITIONS;
    }

    public Map<String, IAttributeDefinition> getAttributeDefinitionMap() {
        return ATTRIBUTE_DEFINITIONS_MAP;
    }
}

```

The above attribute definition will appear under the admin console's login module where the login module handler (implementation) is "RadiusLogin".

Attribute definition supports the following methods, which affect the look and feel of the login module displayed in the Admin Console:

- `setFloor(int)` – automatic validation checking of lower bound integer value
- `setCeiling(int)` – automatic validation checking of upper bound integer value
- `setChoiceList(String[])` – a drop down list with choice values will be rendered
- `setDefaultValue(Object)` – must be the same type of object as declared
- `setInternalUseOnly(Boolean)` – attribute will not be displayed

In order to retrieve the attribute value set in the Admin Console, your login module handler must perform the following:

Save the login module passed in during the initialize call:

Java

```

public void initialize(IServerContext serverContext,
IBaseObject module) {

```

```
this.serverContext = serverContext;
this.loginModule = (ILoginModule) module;
```

Retrieve the attribute value (this sample uses the radius login shown above):

Java

```
String radiusServer =
loginModule.getAttributeStringValueWithDefault(ATTR_RadiusServerIP);
int radiusPort =
loginModule.getAttributeIntegerValueWithDefault(ATTR_RadiusServerPort);
String sharedSecret =
loginModule.getAttributeStringValue(ATTR_RadiusSharedSecret);
```

Sample

Although this sample contains usage of other interfaces in the AccessMatrix server, it is only meant to demonstrate a realistic implementation sample. It is not expected that developers must use these interfaces.

Ldap Login sample for AD adapted from server's default implementation

Java

```
package com.isprint.am.core.auth;

import com.isprint.am.core.IAttributeDefinition;
import com.isprint.am.core.IBaseObject;
import com.isprint.am.core.ISession;
import com.isprint.am.core.IUser;
import com.isprint.am.core.IUserStore;
import com.isprint.am.core.IUserStoreHandler;
import com.isprint.am.core.IModuleFeature.IVerifyPassword;
import com.isprint.am.core.impl.AttributeDefinition;
import com.isprint.am.dto.AttributeConstant;
import com.isprint.am.dto.bean.AttributeTO.DataType;
import com.isprint.am.dto.bean.AttributeTO.EncryptionFlag;
import com.isprint.am.dto.bean.EnumConstant.ObjectClass;
import com.isprint.am.dto.exception.AuthenticationException;
import com.isprint.am.dto.exception.QueryException;
import com.isprint.am.infra.ADLdapUserStore;
import com.isprint.am.infra.LdapConnector;
import com.isprint.am.infra.LdapUserStore;
import com.isprint.am.server.IServerContext;
import java.util.Map;
import javax.naming.NamingException;
import javax.naming.directory.DirContext;
import org.apache.log4j.Logger;

/**
```

Ldap Login sample for AD adapted from server's default implementation

```
* Implements LDAP authentication.
*/
public class LdapLogin implements ILoginHandler, IVerifyPassword {
    static Logger logger = Logger.getLogger(LdapLogin.class);

    private LoginModule loginModule;
    private IServerContext serverContext;
    private String storeId;

    public void initialize(IServerContext serverContext, IBaseObject module)
throws QueryException {
        this.serverContext = serverContext;
        this.loginModule = (LoginModule)module;
        this.storeId = loginModule.getUserStore().getId();
    }

    public void authenticate(SessionContext sessionCtx, ICredential context,
Map<String, String> params) throws AuthenticationException {
        this.verifyPassword(sessionCtx.getSession(), sessionCtx.getUser(),
context, params);
    }

    public String getDN(String uuid) {
        IBaseObject bo =
serverContext.getQueryService().findObjectByUUID(serverContext.getServerSession(), uuid, null);
        if (bo != null) {
            if (bo instanceof IUser)
                return bo.getDN();
            logger.error("Object found, but it is not an instance of IUser,
uuid="+uuid+", baseObject="+bo+", dn="+bo.getDN());
        }
        return null;
    }

    public void verifyPassword(ISession userSession, IUser user, ICredential
credential, Map<String, String> params) throws AuthenticationException {
        String userUUID = user.getUUID();
        String password = credential.getPassword().getClearValue();
        String dn = null;

        try {
            dn = getDN(userUUID);
            if (dn==null)
                throw new AuthenticationException.UserNotFound("Could not
find LDAP DN for user "+user.getId());
            LdapUserStore store = getUserStoreHandler();
            DirContext dc = LdapConnector.getInitialDirContext(store, dn,
password);
            dc.close();
            return;
        } catch (NamingException e) {
            logger.warn("Could not bind user " + dn+ " to LDAP user store
"+storeId);
            if (e instanceof javax.naming.AuthenticationException) {
                // handle AD's return code
                String errorMsg = e.getMessage();
            }
        }
    }
}
```

Ldap Login sample for AD adapted from server's default implementation

```
        logger.warn(e.getMessage());
        // 52e = invalid user id or password, since lookup is
successful, it must be wrong password
        if (errorMsg.indexOf("data 52e")>-1)
            throw new
AuthenticationException.WrongPassword(errorMsg);
        //530 - time restriction
        if (errorMsg.indexOf("data 530")>-1)
            throw new
AuthenticationException.DeniedByPolicy(errorMsg);
        if (errorMsg.indexOf("data 533")>-1)
            throw new
AuthenticationException.AccountNotEnabled(errorMsg);
        if (errorMsg.indexOf("data 775")>-1)
            throw new
AuthenticationException.DeniedByPolicyMaxFailedAttemptsReached();
        if (errorMsg.indexOf("data 701")>-1)
            throw new
AuthenticationException.AccountTimeInvalid(errorMsg);
    } else
        logger.error(e);
        throw new AuthenticationException.OtherReasons(e.getMessage());
    }
}

    public LdapUserStore getUserStoreHandler() throws
AuthenticationException {
        IUserStoreHandler handler =
serverContext.getUserStoreHandler(storeId);
        if (handler instanceof LdapUserStore)
            return (LdapUserStore) handler;
        logger.info("UserStore handler is not found or not an instance of
LdapUserStore, id="+storeId+", handler="+handler);
        throw new AuthenticationException.SystemFailure("UserStore handler
is not found or not an instance of LdapUserStore, id="+storeId);
    }

    public final static IAttributeDefinition _DEF_REQUIRE_USERSTORE = new
AttributeDefinition(AttributeConstant.LoginModule._ATTR_REQUIRE_USER_STORE,
DataType.BOOLEAN, EncryptionFlag.NO, false).setDefaultValue(Boolean.TRUE);
    public static final IAttributeDefinition[] ATTRIBUTE_DEFINITIONS = new
IAttributeDefinition[] {
        _DEF_REQUIRE_USERSTORE,
    };

    public final static Map<String, IAttributeDefinition>
ATTRIBUTE_DEFINITIONS_MAP =
AttributeDefinition.arrayAsDefinitionMap(ATTRIBUTE_DEFINITIONS);
    public IAttributeDefinition[] getAttributeDefinitions() {
        return ATTRIBUTE_DEFINITIONS;
    }

    public Map<String, IAttributeDefinition> getAttributeDefinitionMap() {
        return ATTRIBUTE_DEFINITIONS_MAP;
    }
}
```

Deployment

There are two ways to deploy a custom login module handler:

1. Register the custom login module handler class in the Admin Console's **Global Setting**.
2. Use an AM5 server-side plugin to register the custom login module handler class.

Register via AdminConsole's Global Setting

The compiled class should be deployed in *WEB-INF/classes* for testing. You may want to package it as a signed jar file in *WEB-INF/lib* for UAT/production use. Any dependent jar files should be deployed in *WEB-INF/lib* (please check for any conflict with existing jar files). After the class/jar files are deployed, restart the server and load your login module handler through the the Admin Console's **Global Setting > Module Implementation** tab. It will try to validate whether the class you keyed in exists before you can save the global setting.

Register via AM5 server-side plugin

See the [Plugin](#) section for more details on the interface to implement and deployment.

Testing

Developers should have their own unit test performed before any actual integration is performed in SIT. Use a XML-RPC or SOAP client to call `AuthenticationService.login` or `loginEx` to test in UAT/production.

Delivery Channel Handler

The delivery channel is part of the event notification framework in AM5. An event can be formatted using a message template and sent via a delivery channel to a receiver. For example, an SMS OTP must be sent out through an SMS gateway. Integration with an SMS gateway may be proprietary; hence, it is necessary to write a custom delivery channel handler.

Interface

Developers should implement the following interfaces:

- `com.isprint.am.core.ModuleFeature.IHasAttributeDefinitions`
- `com.isprint.am.core.IModuleHandler.IDeliveryChannelHandler`

Example implementation of interface method for IDeliveryChannelHandler:

Java

```
public MessageDeliveryResult send(IMessageContext
messageContext) {
    StringBuilder sb = new StringBuilder();
    Map<String, Object> map =
messageContext.getMessage().getFields();
    Set<String> keySet = map.keySet();
    for (String key:keySet) {
        sb.append("Key="+key+",
value="+map.get(key)+"\n");
    }
    // do the actual sending to external message
gateway
    log.info("Send message now:\n"+sb.toString());
    MessageDeliveryResult r = new
MessageDeliveryResult();

    r.setStatus(MessageDeliveryResult.STATUS_delivered);
    return r;
}
```

MessageDeliveryResult.setStatus can have the following values:

- "MessageDeliveryResult.STATUS_delivered"
- "MessageDeliveryResult.STATUS_sentAndPending"
- "MessageDeliveryResult.STATUS_failedInDelivery"
- "MessageDeliveryResult.STATUS_failedInCreatingMessage"

You should also copy the result code/message sent back by external message gateway by using the following methods in MessageDeliveryResult:

- setExtendedResultCode(String resultCode);
- setResultMessage(String msg);
- setCustomAudit(String customName, String value);

Initialization

All thread-safe initialization should be done inside the initialize method. The initialize method will be called when the login module handler is first used. Developers should log initialization information under INFO or ERROR logging in **am.log**.

Logging

Developers should use the log4 logger used by the AccessMatrix server. By default, the log will go to **am.log**. However, a different logger can be configured via *WEB-INF/classes/amlog4j.properties* if necessary:

To use the log4j logger, declare a private static variable in your implementation class with the class name used in the getLogger parameter:

Java

```
private static final Logger log =  
    LogManager.getLogger(MyDeliveryChannelHandler.class);
```

Deployment

For deployment, please refer to the [Plugin](#) section on how to:

- Use a plugin to register a custom delivery channel handler.
- Deploy the plugin.

3.3. Custom Response Collector

Authentication Use Case

Authentication

The developer is assumed to be familiar with the concept of authentication realms. They should also know how to call the AM5 web service API (XML-RPC or SOAP) and configure the authentication realm to point to a `ResponseConfiguration` object to return a custom response upon successful authentication. The custom response may be used to perform access control enforcement (e.g. in the portal server).

This use case is supported as long as developer has the UAS or UAM licensing.

Interface

com.isprint.am.core.CustomResponseCollector	
Java	<pre>package com.isprint.am.core; import java.util.Map; /** * Custom response collector. This allows partner, client or * PS developer to develop a class deployed within AccessMatrix * server's WEB-INF/classes or WEB-INF/lib to return a * custom response string for a certain response key configured * under the ResponseConfiguration * in admin console where the attribute name of the response * key starts with "@" sign sign followed by the implementation * class e.g. * if the implementation class is * "com.myco.MyCustomResponseCollector" then the attribute name * specified as attribute name in * ResponseConfiguration for that response key is * "@com.myco.MyCustomResponseCollector". The implementation * class will then be loaded by AccessMatrix runtime and the * getResponseString method will be called. * * For thread-safe initialization (e.g. creating JNDI data * source), developer should put it under static * initializer in their implementation class. Initialization * should not be done in getResponseString method * * @author jitkiatt * @see SampleCustomResponseCollector */</pre>

com.isprint.am.core.CustomResponseCollector

```
public interface CustomResponseCollector {

    /**
     * Get the response string for the response key specified
     * by responseKey
     * @param user the authenticated user object, user
     * id/uuid/name/dn might as a search key to external system.
     * @param responseKey the key used in the response
     * @param params reserved for future use
     * @return a String to as value of the response key
     */
    String getResponseString(IUser user, String responseKey,
        Map<String, Object> params);

}
```

Initialization

Developers should use a static initializer to create a thread-safe one-time only initialization code e.g. (establish JNDI data source or create connection pool, etc.).

Example:

Java

```
private static DataSource ds;
// static initializer
static {
    log.info("Initializing
SampleCustomResponseCollector");

    String dsName = "java://comp/env/jdbc/DSamdb"; //
should not use DSamdb, sample only
    try {
        Context ctx = new InitialContext();
        ds = (DataSource) ctx.lookup(dsName);
    } catch (Exception e) {
        log.error("Failed to get datasource "+dsName+",
msg="+e.getMessage(),e);
        // do not throw exception if you still want the
authentication/authz to go through when this data
source is not available,
        //throw new
SystemException.DatabaseException("Failed to get
datasource "+dsName+", msg="+e.getMessage(),e);
    }
}
```

Logging

Developer should use the log4 logger used by the AccessMatrix server. By default, the log will go to **am.log**. However, a different logger can be configured via the *WEB-INF/classes/amlog4j.properties* if necessary:

To use the log4j logger, declare a private static variable in your implementation class with the class name used in the getLogger parameter:

Java

```
private static final Logger log =  
    LogManager.getLogger(YourCustomResponseCollector.class);
```

Sample Configuration in Admin Console

Edit the User Response:

1. Navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response** and click on the "GroupMembership" User Response.
2. Add the custom attribute "sampleCustomResponse".
3. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and click on the authentication realm you want to use.
4. Under **User Response**, update the value to "GroupMembership".
5. Click **Save**.

Sample Implementation

Java

```
import java.sql.Connection;  
import java.sql.PreparedStatement;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.util.Map;  
  
import javax.naming.Context;  
import javax.naming.InitialContext;  
import javax.sql.DataSource;  
  
import org.apache.log4j.LogManager;
```

```
import org.apache.log4j.Logger;

import com.isprint.am.core.CustomResponseCollector;
import com.isprint.am.core.IUSER;

public class SampleCustomResponseCollector implements
CustomResponseCollector {
    private static final Logger log =
LogManager.getLogger(SampleCustomResponseCollector.class);
    private static DataSource ds;
    // static initializer
    static {
        log.info("Initializing
SampleCustomResponseCollector");

        String dsName = "java://comp/env/jdbc/MyAppDB";
        try {
            Context ctx = new InitialContext();
            ds = (DataSource) ctx.lookup(dsName);
        } catch (Exception e) {
            log.error("Failed to get datasource
"+dsName+", msg="+e.getMessage(),e);
            // do not throw exception if you still want
the authentication/authz to go through when this data
source is not available
        }
    }
    // example to return "0,GROUP1,GROUP2,GROUP3" where
GROUP1-GROUP3 are external custom groups stored in MyAppDB
    public String getResponseString(IUser user,String
responseKey, Map<String,Object> params) {
        String result = "0,RESP";
        // in actual sample, most likely you will need
these values
        String userId = user.getId();
        String userUUID = user.getUUID();
        String userStoreId = user.getUserStore().getId();
        if (ds != null) {
            // your sql might include user id or user uuid
in the where clause
            // fire your SQL here to your remote DB to and
massage it to set to the result
            // this sample use the same DB AM5 is using

            try {
                Connection conn = ds.getConnection();
                // CHANGE ME, a silly sample to return
count of table
                PreparedStatement ps =
conn.prepareStatement("select count(*) from myTable");
                ResultSet rs = ps.executeQuery();
                rs.next();
                result = "0,"+rs.getInt(1);
            } catch (SQLException e) {
                log.error(e);
                result = e.getErrorCode()+e.getMessage();
            }
        }
    }
}
```

```
// set error code, message in result
    }
    } else {
        result = "1,DATA_SOURCE_NOT_AVAILABLE";
    }
    return result;
}
}
```

Sample Client Test Code

XML-RPC (Java)

Java

```
ApiProxy proxy = null;
Map<String, String> params = new HashMap<String, String>();

params.put("realmId", "40172"); // AD login
try {
    proxy = ApiProxyFactory.createProxy("SimpleFactory");
    ModifyUser modUser = new ModifyUser("ActiveDirectory",
    "", "userid");
    modUser.assignRealm("40172", null, null);
    proxy.getUpdateService().execute(adminSesToken, modUser,
    new HashMap());
    AuthResultTO result = proxy.getAuthnSvc().login("",
    "userid", "password", params);
    assertTrue(result.getSession().isAuthenticated());
    logger.info(result.getResponses().toString());
} catch (XmlRpcException e) {
    fail(e.code + ", " + e.getMessage());
} finally {
    proxy.close();
}
```

Sample System Output

```
memberOf=i-Sprint,Product Engineering,Professional Service, status=Activated (512),
lastSuccessfulLogin=1286361337388, sampleCustomResponse=100
```

.Net

C#

```
ModifyUser modUser = new ModifyUser("ActiveDirectory", "",
"sandar");
modUser.assignRealm("40172", DateTime.MinValue,
DateTime.MinValue); //MinValue is used to represent null
value
```

.Net

```
pxy.UpdateSvc.execute(getAgentSessionToken(), modUser, new
XmlRpcStruct());
sb.Append("Successfully assigned Realm Id to AD User!");

protected static void
testNTLMLoginWithADGroupMembershipAsResponse()
{
    Proxy pxy = null;
    AuthResult rs = null;
    try
    {
        pxy = getProxy();
        Console.WriteLine("Please Enter UserId = ");
        string userId = Console.ReadLine().Trim();
        Console.WriteLine("Please Enter Password/Credential =
");
        string credential = Console.ReadLine().Trim();

        XmlRpcStruct parms = new XmlRpcStruct();
        parms["realmId"] = "40172";
        StringBuilder sb = new StringBuilder();
        sb.Append("
User Login: ");
        rs = pxy.AuthnSvc.login("", userId, credential,
parms);
        if (rs != null && rs.session.authenticated)
        {
            userSessionToken = rs.session.sessionToken;
            userUUID = rs.session.userUUID;

            sb.Append("
Login/1FA Authentication is successful!" + "
You are logged in as user: " + userId);
            loginUserId = userId;
            sb.Append("
Login User's UUID: " + userUUID);
            if (rs.responses != null && rs.responses.Count
>= 1)
            {
                sb.Append("
Getting User's Attributes from Response Configuration: ");
                XmlRpcStruct respEntries = rs.responses;
                sb.Append("
\t . Response Entries are as follows:");

                //To retrieve the value directly, any key
name that was configured in Response Configuration can be
specified without using IEnumerator to dump out all
contents.

                //string lastLogonAD =
respEntries["lastLogonAD"].ToString();
                //sb.Append("
\t - " + " lastLogonAD = " + lastLogonAD);

                IEnumerator keys =
respEntries.Keys.GetEnumerator();
                while (keys.MoveNext())
```

.Net

```
        {
            object key = keys.Current;
            sb.Append("
\t - " + key + " = " + respEntries[key]);
        }
    } else sb.Append("
Getting Response Entries is null!");
    LoginModuleState[] loginModStates =
rs.loginModuleStates;
    ArrayList stateList = new ArrayList();
    for (int i = 0; loginModStates != null && i
< loginModStates.Length; i++)
    {
        if
(loginModStates[i].forceChangePassword)
            stateList.Add(loginModStates[i]);
    }
    if (stateList.Count > 0)
    {
        sb.Append("
You are being forced to change password!");
    }
    else sb.Append("
Login Failed!");
    Console.WriteLine(sb.ToString());
}
catch (XmlRpcFaultException xmle)
{
    Console.WriteLine("XmlRpcFaultException: " +
"FaultCode = " + xmle.FaultCode + ";
FaultString = " + xmle.FaultString + ";
Exception Message = " + xmle.Message);
}
catch (WebException svre)
{
    Console.WriteLine("WebException: " + "
Message = " + svre.Message);
}
catch (Exception ex)
{
    Console.WriteLine("Exception: " + "
Message = " + ex.Message + ";
Source = " + ex.Source + ";
StackTrace = " + ex.StackTrace);
}
finally
{
    if (pxy != null)
        pxy.Close();
}
}
```

SOAP (Java)

Java

```
@WebServiceRef(wsdlLocation="http://localhost:8080/am5/soap/auth?wsdl")
static AuthenticationService_Service authService;
static AuthenticationService asPort;
static String userStore = "DefaultStore";
static Properties props = null;

static {
    try {

        authService = new AuthenticationService_Service();
        logger.info("Retrieving the port from the following service: "
+ authService);
        asPort = authService.getAuthenticationServicePort();

    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void login() {
    try {
        SoapMap params = new SoapMap();
        MapEntry remoteAdd = new MapEntry();
        remoteAdd.setKey("remoteAddr");
        remoteAdd.setValue("192.168.1.67");
        params.getEntries().add(remoteAdd);
        MapEntry remoteAdd2 = new MapEntry();
        remoteAdd2.setKey("realmId");
        remoteAdd2.setValue("40151");
        params.getEntries().add(remoteAdd2);

        AuthResultTO result = asPort.login("", "demouser", "password1",
params);
        logger.info("result's
response:"+result.getResponses().toString());

        if(result.getResponses() != null) {
            for (BasicTypeMapEntry c:
result.getResponses().getEntries()) {
                if(c.getKey()!=null) {
                    logger.info("result's response,
key="+c.getKey()+" , value="+c.getValue().getStringValue()+" ; ");
                }
            }
        }

        if (result != null) {
            if(result.getSession() != null) {
                SessionTO s = result.getSession();
            }
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

SOAP (Java)	
<pre>} }</pre>	

Access Control Use Case

Access Control

This is a more advanced use case requiring UAM license where the custom response will be either:

1. Injected into the HTTP header by WSA if WSA is used.
2. Retrieved through the `AuthorizationService` API.

Explanation of this use case is beyond the scope of this document. To fully understand this use case, developers should engage i-Sprint Innovations or one of our authorized partners for training.

3.4. Non-Blocking Event Listener

AccessMatrix 5 has a built-in event notification framework where event can be sent out via SMTP mail, HTTP, SMS or SMPP message. However, it is also possible to write custom event listener to listen to an event within the AM 5 server in a non-blocking manner. An example will be to listen to authentication events for billing purposes, or to send violation events to SIEM (Security Information and Event Management) tools. The event listener is expected to be implemented by our PS team or our qualified/certified professionals only. It is not expected to be implemented by client developers directly.

Interface

The implementation class must implement the interface `com.isprint.am.core.IEventListener` which has the following method:

<code>com.isprint.am.core.IEventListener</code>	
Java	
<pre>public void eventReported(com.isprint.am.core.event.IEvent event);</pre>	

The event category / types are listed in:

- `com.isprint.am.dto.bean.EnumConstant.EventCategoryTO` (e.g. Principal)
- `com.isprint.am.dto.bean.EnumConstant.EventTypeTO` (e.g. AddUser)

So user creation event category / type will be Principal-AddUser

Note you can further filter the event by event result and result code. The list of result code is available in the XML RPC API guide in the Appendix. For example, for the Authentication-Login event type, the result code 10403 is for the max failed attempts reached for authentication, whereas 10105 means the account is not enabled.

You can get the event result by calling the `getResult()` method from the `IEvent` interface and compare it against the following enum:

- `com.isprint.am.core.event.EventResult.SUCCESSFUL`
-

-
- `com.isprint.am.core.event.EventResult.FAILED`

You can get the result code by calling the `getResultCode()` method from the `IEvent` interface and compare it against the result code documented in *AM5 XML-RPC and SOAP API Reference Appendices*.

You can also refer to the default message template Help dialogue to check what are the other available event information such as extended result code and event message. You can also examine the output of `event.dumpAsString()` to see what are the event fields available through the `event.getFieldXXX()` method.

Initialization

Developer should use the `IPlugin` interface's `load` method to initialize the event listener.

Logging & Error Handling

Developer should use the `log4j` logger used by the AccessMatrix server. By default, the log will go to **am.log**. However, a different logger can be configured via the *WEB-INF/classes/amlog4j.properties* if necessary:

To use the `log4j` logger, declare a private static variable in your implementation class with the class name used in the `getLogger` parameter:

Java

```
private static final Logger log =  
    LogManager.getLogger(MyEventListener.class);
```



Important Note: Developers should catch all throwables and log the exception stack trace within the Event Listener class, or else there will not be a default handler for exception.

Event Listener Registration

Developer must register his custom event listener in the `plugin load()` method to the event manager and specify the event category as well as the event type. See the sample code below:

Sample Code

SampleAuthnEventListener

SampleAuthnEventListener

Java

```
import org.apache.log4j.Logger;

import com.isprint.am.core.IEventListener;
import com.isprint.am.core.event.EventCategoryTypePair;
import com.isprint.am.dto.bean.EnumConstant.EventCategoryTO;
import com.isprint.am.dto.bean.EnumConstant.EventType;
import com.isprint.am.server.IPlugin;
import com.isprint.am.server.IPluginContext;

public class SampleAuthnEventListenerPlugin implements
IPlugin {
    private static final Logger logger =
Logger.getLogger(SampleAuthnEventListenerPlugin.class);
    SampleAuthnEventListener listener = null;
    public String getId() {
        return "MyEventListener";
    }

    public String getVersion() {
        return "1.0";
    }

    @Override
    public void load(IPluginContext pc) {
        // first we register ourselves with the server's
event manager to listen to login/reauthentication events
        listener = new SampleAuthnEventListener();

pc.getServerContext().getEventService().addEventListener(
        new EventCategoryTypePair[] {
            EventCategoryTypePair.valueOf(
                EventCategoryTO.Authentication,
                EventType.Authentication.Login),
            EventCategoryTypePair.valueOf(
                EventCategoryTO.Principal,
                EventType.Principal.AddUser)
        }, listener
    );
    logger.info("Loaded " + getId() + " - " +
getVersion());
}

    @Override
    public void unload(IPluginContext pc) {
        // unload your JNDI data source or do clean up

pc.getServerContext().getEventService().removeEventListener(
        new EventCategoryTypePair[] {
            EventCategoryTypePair.valueOf(
                EventCategoryTO.Authentication,
                EventType.Authentication.Login),
            EventCategoryTypePair.valueOf(
                EventCategoryTO.Principal,
                EventType.Principal.AddUser)
        })
    }
```

SampleAuthnEventListener

```
        }, listener
    };
    logger.info("UnLoaded " + getId() + " - " +
        getVersion());
    }
}
```

SampleAuthnEventListenerPlugin

Java

```
import org.apache.log4j.Logger;

import com.isprint.am.core.IEventListener;
import com.isprint.am.core.event.EventCategoryTypePair;
import com.isprint.am.dto.bean.EnumConstant.EventCategoryTO;
import com.isprint.am.dto.bean.EnumConstant.EventType;
import com.isprint.am.server.IPlugin;
import com.isprint.am.server.IPluginContext;

public class SampleAuthnEventListenerPlugin implements
IPlugin {
    private static final Logger logger =
        Logger.getLogger(SampleAuthnEventListenerPlugin.class);
    SampleAuthnEventListener listener = null;
    public String getId() {
        return "MyEventListener";
    }

    public String getVersion() {
        return "1.0";
    }

    @Override
    public void load(IPluginContext pc) {
        // first we register ourselves with the server's
        // event manager to listen to login/reauthentication events
        listener = new SampleAuthnEventListener();

        pc.getServerContext().getEventService().addEventListener(
            new EventCategoryTypePair[] {
                EventCategoryTypePair.valueOf(
                    EventCategoryTO.Authentication,
                    EventType.Authentication.Login),
                EventCategoryTypePair.valueOf(
                    EventCategoryTO.Principal,
                    EventType.Principal.AddUser)
            }, listener
        );
        logger.info("Loaded " + getId() + " - " +
            getVersion());
    }

    @Override
```

SampleAuthnEventListenerPlugin

```
public void unload(IPluginContext pc) {
    // unload your JNDI data source or do clean up

    pc.getServerContext().getEventService().removeEventListener(
        new EventCategoryTypePair[] {
            EventCategoryTypePair.valueOf(
                EventCategoryTO.Authentication,
                EventType.Authentication.Login),
            EventCategoryTypePair.valueOf(
                EventCategoryTO.Principal,
                EventType.Principal.AddUser)
        }, listener
    );
    logger.info("UnLoaded " + getId() + " - " +
        getVersion());
}
```

Deployment

The custom class must be packed into a jar file and placed in the <ACMATRIX_HOME>/plugins/ directory, where <ACMATRIX_HOME> is the app server's current directory. The name of the jar must match what you return in getId() method in IPlugin interface. If your plugin id is MyEventListener then name the jar file **MyEventListener.jar**.

The plugin directory is also configurable in **amsystem.properties** using the following property:

Properties (.properties files)

```
am.pluginindir
```

Note it is optional to use a plugin property file if you need to enable/disable or control the loading order of the plugin. Possible content of the property file:

Properties (.properties files)

```
# the following must match the value returned by
getId() in IPlugin interface
plugin.id=MyEventListener
plugin.MyEventListener.enabled=true
# load order
plugin.MyEventListener.loadOnStartup=3
```

3.5. Plugin Service

As a precondition to writing a server plugin service extension, the Professional Services team or our partner must be technically competent in writing server-side plugins, as well as trained and certified in server-side development by i-Sprint Innovations.

The objectives of having an XML-RPC extension are described in the following sub-sections.

Transaction Support

By default, XML-RPC API does not support transactions. For example, you cannot combine Create User and Reset Password into one transaction even though both of them may involve the same database. For certain projects that requires transactional support, PS/partner can write a server-side plugin that make use of an XML-RPC extension generic method to combine multiple operations on the server-side into the same transaction.

Custom Function

Certain project may require customized API not currently supported by AccessMatrix product. For example, it may require a custom table created within the AccessMatrix default store with a custom function built by PS/partner. This custom function may need to be accessed remotely by using the XML-RPC web service. Therefore, a general extension mechanism must be available to fulfill such a custom function.

Interfaces

IPluginService Interface

The implementation class must implement the interface `com.isprint.am.server.IPluginService`, which is a subclass of the `com.isprint.am.server.IPlugin` interface and has the following methods:

com.isprint.am.server.IPlugin	
Java	
<pre>public String getId() ; public String getVersion() ;</pre>	

com.isprint.am.server.IPlugin	
<pre>public void load(IPluginContext pc); public void unload(IPluginContext pc);</pre>	
com.isprint.am.server.IPluginService	
Java	<pre>/** * @param cx serverContext to get various domain layer services if required * @param session the user login session restored from session token sent by web service client * @param args it is recommended that args[0] will contain the method (operation) name and the rest contain * method parameters * @return this will depend on which use case * @throws com.isprint.dto.exception.BaseException which will be translated as XmlRpcException back to the client */ Object execute(IServerContext cx, ISession session, Object[] args) throws BaseException;</pre>

Client Interface

A client application may invoke the plugin service via XML-RPC API or REST API (supported from AccessMatrix 5.5.1-GA onwards).

XML-RPC API

The plugin service is called via the method below in Data Protection Service:

Java
<pre>Object executePluginService(String sessionToken, String pluginId, Object[] arguments, Map params)</pre>

Returns:

Whatever is returned by the server plugin service (provided it is a supported XML-RPC data type).

Method arguments:

-
- **sessionToken**: a logged in session ID
 - **pluginID**: the plugin ID that must match the one in the target server plugin service
 - **arguments**: a list of XML-RPC supported data values that will be passed to the server plugin service `execute()` method
 - **params**: reserved for additional parameters (just pass an empty Map)

REST API

The plugin service is called via REST API through the URL: `/rest/plugin/<PluginID>`

JSON Request
JSON <pre>{ "sessionToken": // the login session token "args": [/* this array will correspond to the same 'args' array parameter of the server-side method */], "params": { /* this JSON object will correspond to the Map 'params' parameter of the server-side method */ } }</pre>
JSON Response
JSON <pre>{ "result": /* the value will be the Object returned by the server-side method marshalled as JSON */ }</pre>

Supported Data Types

The data types must be supported for both XML-RPC and JSON.

For XML-RPC, the actual types used depends on choice of programming language and XML-RPC Library.

XML-RPC Type	Apache XML-RPC Library for Java	CookComputing XML-RPC Library for Microsoft .NET	JSON type
int or i4	Int, Integer	Int32, int	number
double	Double	Double	number
Boolean	Boolean	Bool	boolean
ISO8601 datetime	Java.util.Date	DateTime	number (long millis)
Base64 (binary)	Byte[]	Byte[]	string (base64 encoded)
Array	Object[]	Object[], Array	JSON array
Struct	Map	XmlRpcStruct	JSON object

Note that arrays and structs may contain values that are of any supported types, including other structs and arrays.

Initialization

Developer should use the IPlugin interface's load method to initialize anything that requires thread safety.

Logging

Developer should use the log4 logger used by the AccessMatrix server. By default, the log will go to **am.log**. However, a different logger can be configured via the *WEB-INF/classes/amlog4j.properties* if necessary:

To use the log4j logger, declare a private static variable in your implementation class with the class name used in the getLogger parameter:

Java

```
private static final Logger logger =
    LogManager.getLogger (HelloWorldService.class);
```

Deployment

The custom class must be packed into a jar file and placed in the <ACMATRIX_HOME>/plugins/ directory, where <ACMATRIX_HOME> is the app server's current directory. The name of the jar must match what you return in the getId() method in IPlugin interface. If your plugin id is HelloWorldService, then name the jar file **HelloWorldService.jar**.

The plugin directory is also configurable in **amsystem.properties** using the following property:

Properties (.properties files)
<code>am.pluginindir</code>

Note that it is optional to use a plugin property file if you need to enable/disable or control the loading order of the plugin. Possible content of the property file:

Properties (.properties files)
<pre># the following must match the value returned by getId() in IPlugin interface plugin.id=HelloWorldService plugin.HelloWorldService.enabled=true # load order plugin.HelloWorldService.loadOnStartup=3</pre>

Sample Code

Server side plugin service code:

HelloWorldService	
Java	<pre>import org.apache.log4j.LogManager; import org.apache.log4j.Logger; import com.isprint.am.core.ISession; import com.isprint.am.dto.exception.BaseException; import com.isprint.am.server.IPluginContext; import com.isprint.am.server.IPluginService; import com.isprint.am.server.IServerContext; import com.isprint.am.util.Preconditions; public class HelloWorldService implements IPluginService { private static final Logger logger =</pre>

HelloWorldService

```
LogManager.getLogger(HelloWorldService.class);
    public String getId() {
        return "HelloWorldService";
    }

    public String getVersion() {
        return "1.0";
    }

    public void load(IPluginContext arg0) {
        logger.info("HelloWorldService init ...");
    }

    public void unload(IPluginContext arg0) { }

    public Object execute(IServerContext ctx, ISession
session, Object[] args)
        throws BaseException {
        Preconditions.ensureParametersNotNull(args[0]);
        String methodName = (String)args[0];
        String whoToGreet = (String)args[1];
        if (methodName.equals("HelloWorld")) {

Preconditions.ensureParametersNotNull(whoToGreet);
            Preconditions.checkEmptyString(whoToGreet, true,
"Expect whoToGreet param for HelloWorld");
            return HelloWorld(whoToGreet);
        } else if (methodName.equals("GoodMorning")) {

Preconditions.ensureParametersNotNull(whoToGreet);
            Preconditions.checkEmptyString(whoToGreet,
true, "Expect whoToGreet param for GoodMorning");
            return GoodMorning(whoToGreet);
        } else if (methodName.equals("howOldAreYou")) {

Preconditions.ensureParametersNotNull(whoToGreet);
            Preconditions.checkEmptyString(whoToGreet,
true, "Expect whoToGreet param for GoodMorning");
            return getAge(whoToGreet);
        } else
            throw new
java.lang.IllegalArgumentException("Methods supported are
only HelloWorld <name> and GoodMorning <name>");

        }

    private String GoodMorning(String whoToGreet) {
        return "Good Morning, "+whoToGreet;
    }

    private String HelloWorld(String whoToGreet) {
        return "Hello World, "+whoToGreet;
    }
}
```

XML-RPC Client Application code

Java

```
/*
 * Invoke the server side plugin service via
 * DataProtectionService
 * executePluginService method.
 */
String pluginId = "HelloWorldService";
// this returns "Good Morning, Gaga"
String s1 =
proxy.getDataProtectionSvc().executePluginService(
    sessionToken, pluginId, new Object[]{"GoodMorning",
"Gaga"}, params);
// this returns "Hello World, MDNA"
String s2 =
proxy.getDataProtectionSvc().executePluginService(
    sessionToken, pluginId, new Object[]{"HelloWorld",
"MDNA"}, params);
// this returns 53
int age = proxy.getDataProtectionSvc().executePluginService(
    sessionToken, pluginId, new Object[]{"howOldAreYou",
"MDNA"}, params);
```

REST API

URL: /rest/plugin/HelloWorldService

Request

Response

JSON

```
{
  "sessionToken": "...",
  args: [
    "GoodMorning",
    "Gaga"
  ]
}
```

JSON

```
{
  "result": "Good Morning, Gaga"
}
```

JSON

```
{
  "sessionToken": "...",
  args: [
    "HelloWorld",
    "MDNA"
  ]
}
```

JSON

```
{
  "result": "Hello World, MDNA"
}
```

REST API	
JSON	JSON
<pre>{ "sessionToken": "...", args: ["howOldAreYou", "MDNA"] }</pre>	<pre>{ "result": 53 }</pre>

3.6. JMS Integration (Message Driven Bean)

In the banking industry, it is common to use EAI (Enterprise Application Infrastructure) e.g. Tuxedo/Oracle Enterprise Service Bus to integrate legacy systems with AccessMatrix UAS/UAM. The delivery protocol is normally IBM MQ. One example is the ATM/IVR system, which may require the generation and verification of SMS OTPs or the verification of token OTPs. Such legacy system may send a request to EAI in the bank for data transformation. EAI will then deliver it to the UAS server via MQ. AccessMatrix UAS/UAM can support such integration via standard JMS Message-Driven Bean to receive the request and reply to EAI.

Supported Containers

JMS/Message Driven Bean require an J2EE Application server/EJB Container. Supported EJB containers are now IBM WebSphere 7.0/Oracle WebLogic 10 and above.

Compilation

Other than **am.jar**, you need MDB specific jar files from the application server vendor. For Oracle WebLogic, the following jar files are required:

- **wlclient.jar**
- **wljmsclient.jar**

Initialization & Cleanup

Initialization can be done within the `ejcCreate` method, Cleanup can be done in the `ejbRemove` method.

Error Handling and Logging

Developer should use the log4j logger used by the AccessMatrix server. By default, the log will go to **am.log**. However, a different logger can be configured via the *WEB-INF/classes/amlog4j.properties* if necessary:

To use the log4j logger, declare a private static variable in your implementation class with the class name used in the `getLogger` parameter:

```
Java
```

```
private static final Logger logger =
    LogManager.getLogger(MyMessageDrivenBean.class);
```

Within the `onMessage` method, developers should try to catch the following exceptions and sent back the AccessMatrix error codes via EAI to the caller.

Java

```
try {
    // do something
} catch (com.isprint.am.dto.exception.BaseException e)
{
    // send back the following to EAI via JMS Message
    logger.error(e);
    e.getCode(); // AccessMatrix error codes
    e.getExtendedCode(); // those returned by HSM or
    Vasco SDK
    e.getMessage()
    e.getExtendedMessage();
} catch
(com.isprint.am.dto.exception.BaseRuntimeException) {
    logger.error(e);
    // send back the following to EAI via JMS Message
    e.getCode(); // AccessMatrix error codes
    e.getExtendedCode(); // those returned by HSM or
    Vasco SDK
    e.getMessage();
    e.getExtendedMessage();
} catch (Exception e) {
    logger.error(e);
    // handle non AccessMatrix errors
}
```

Sample Code

Java

```
package com.isprint.amcust.msgbean;

import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

import com.isprint.amcust.bean.RequestTO;
import com.isprint.amcust.service.IvrSmsOtpService;

import org.apache.log4j.Logger;
```

```
import javax.ejb.*;
import javax.naming.*;
import javax.jms.*;

public class SmsOtpMdb implements MessageDrivenBean,
MessageListener {
    private MessageDrivenContext ctx = null;
    private static Logger logger =
Logger.getLogger(SmsOtpMdb.class);
    Properties props = new Properties();
    String queueName;
    String factoryName;
    String propertiesFileName =
"ivrmessage.properties";

    public SmsOtpMdb() { }

    public void
setMessageDrivenContext(MessageDrivenContext ctx)
        throws EJBException {
        this.ctx = ctx;
    }

    public void ejbCreate() throws CreateException {
        try {
            ReadProperties();
            queueName =
props.getProperty("QUEUE_NAME");
            factoryName =
props.getProperty("FACTORY_NAME");
            logger.info("IVR MDB EJB Created. ");

        } catch (IOException e) {
            logger.error(e);
        }
    }

    public void ejbRemove() {
        logger.info("IVR MDB EJB is removed");
    }

    public void onMessage(Message message) {
        try {

            // Get the message, here we could have an
            ObjectMessage containg
            // an object of a known class. We use Text
            here for simplicity
            if (message instanceof TextMessage) {
                TextMessage m = (TextMessage) message;
                logger.debug("Message read: " +
m.getText());

                String xmlRq = m.getText();
                RequestTO req =
RequestTO.ReadXML(xmlRq);
                logger.info("MQ message "+
req.getSvcRqId() + " received");
            }
        }
    }
}
```

```
        String responseString = "";
        IvrsmsOtpService service = new
IvrSmsOtpService(props);
        responseString =
service.process(req).getResponseXmlString();

        writeResponseToJMSQueue(responseString,
m);

        logger.info("MQ message "+
req.getSvcRqId() + " response sent");
    }
    } catch (JMSException je) {
        logger.error("JMS Exception :::: " +
je.getMessage());
        logger.error("JMS linked Exception :::: " +
je.getLinkedException());
        je.getLinkedException().printStackTrace();
    } catch (Exception e) {
        logger.error(e);
    }
}

private void writeResponseToJMSQueue(String
responseString, TextMessage m) throws NamingException,
JMSException{
    // write back to queue
    QueueSession jmsSession = null;
    QueueSender queueSender = null;

    InitialContext ic = new InitialContext();

    QueueConnectionFactory jmsConnectionFactory =
(QueueConnectionFactory) ic.lookup(factoryName);
    logger.info("looked up QueueConnectionFactory:
" + jmsConnectionFactory);

    Queue queue = (Queue)ic.lookup(queueName);
    logger.info("looked up Queue: " +queue);

    QueueConnection connection =
jmsConnectionFactory.createQueueConnection();
    logger.info("created queue connection::::" +
connection);

    jmsSession =
connection.createQueueSession(false,
Session.AUTO_ACKNOWLEDGE);
    queueSender = jmsSession.createSender(queue);
    TextMessage rpMessage =
jmsSession.createTextMessage();

    rpMessage.setText(responseString);

    rpMessage.setJMSCorrelationID(m.getJMSMessageID());

    logger.info("Request Message ID: " +
m.getJMSMessageID());
```

```

        logger.info("Sending message: " +
rpMessage.getText());

        queueSender.send(rpMessage);

        logger.info("Response messageid : " +
rpMessage.getJMSMessageID());
        logger.info("Response Message Correlation ID: "
+ rpMessage.getJMSCorrelationID());
    }

    private void ReadProperties() throws IOException {
        try {
            InputStream propertiesFile =
this.getClass().getClassLoader()

.getResourceAsStream(propertiesFileName);
            if (propertiesFile != null) {
                props.load(propertiesFile);
            }
        } catch (IOException ioe) {
            logger.info("Cannot load properties file: "
+ propertiesFileName);
        }
    }
}

```

Deployment

It is necessary to repackage the original AM5 WAR into an Enterprise Archive (EAR) file. Sample EAR file for WebLogic Server:

Code

```

/APP-INF
    /classes (moved from am5.war/WEB-INF/)
    /lib (moved from am5.war/WEB-INF/)
/META-INF
    /application.xml (see attachment)
/mdb.jar
    /com.isprint.amcust.... (mdb classes)
/META-INF
    /ejb-jar.xml
    /weblogic-ejb-jar.xml
/am5.war

```

3.7. User Auto-Provisioning for SAML Service Provider

You can implement the `com.isprint.am.core.saml.ICreateNewUser` interface to allow customized user creation. It can be used for the automatic provisioning of users in the AccessMatrix server when AccessMatrix acts as the SAML Service Provider (SP). Users are provisioned upon the successful parsing of the SAML Response.

Customized plugins developed using this interface can be used with the following SAML lookup modules for user auto-provisioning:

- [SAML artifact resolution lookup and login](#)
- [SAML web browser SSO lookup and login](#)

This interface has the following methods:

com.isprint.am.core.saml.ICreateNewUser	
Java	
<pre>public IUser createNewUser(SAMLResponse samlResponse, Map<String,String> params);</pre>	
Input Parameters	
• <code>samlResponse</code> - The resolved SAML Response that contains the user identity and assertion attributes. • <code>params</code> - The params passed in via the Login API, which may contain extra information required for the mapping or other customized uses. • <code>storeId</code> - The Id of the user store that the user will be created in. • <code>userUUID</code> - The UUID of the user being created in the user store.	

Sample Code

The following is a sample code for a plugin that implements the `IPlugin` and `ICreateNewUser` interfaces in order to achieve user auto-provisioning in the AccessMatrix server from a SAML Response.

autoProvPlugin.java

autoProvPlugin.java

Java

```
package plugin;

import java.util.Map;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import com.isprint.am.core.saml.ICreateNewUser;
import com.isprint.am.core.saml.SAMLResponse;
import com.isprint.am.server.IPlugin;
import com.isprint.am.server.IPluginContext;

public class autoProvPlugin implements IPlugin, ICreateNewUser {

    private static final Logger log =
LogManager.getLogger(autoProvPlugin.class);
    public autoProvPlugin() {}

    @Override
    public String getId() {
        return autoProvPlugin.class.getSimpleName();
    }

    @Override
    public String getVersion() {
        return "0";
    }

    /**
     * Called when a <code>Server</code> loads this plugin. To
     unload the plugin <code>Server</code> calls unload method.
     * <p> All plugin class must have a default constructor.
     * The default constructor by definition takes no input
     parameters, constructs the plugin instance, but should not perform
     initialization yet. The initialiation code that modifies the plugin
     state and variables should be done in load.
     * <p> Plugin should allow the load method after it is
     unloaded. This happens when <code>Server</code> is requested to
     reload the plugin and when executing plugin auto-refresh.
     */
    @Override
    public void load(IPluginContext pc) {
pc.getServerContext().addRegisteredClass(ICreateNewUser.class,
this.getClass());
    }

    /**
     * Called when a <code>Server</code> unloads this plugin. The
     plugin had previously been loaded by <code>Server</code> by calling
     load method.
     * <p> Plugin should allow the load method after it is
     unloaded. This happens when <code>Server</code> is requested to
```

autoProvPlugin.java

```
reload the plugin and when executing plugin auto-refresh.
    */
    @Override
    public void unload(IPluginContext pc) {

pc.getServerContext().removeRegisteredClass(ICreateNewUser.class,
this.getClass());
    }

    /**
     * Create a new User based on SAMLResponse (from SAML Artifact
     Response) and the passed in params of Login API.
     *
     * @param samlResponse
     * @param params
     * @param storeId
     * @return userUUID
     */
    public String createNewUser(SAMLResponse samlResponse,
Map<String,String> params, String storeId) {
        log.error("autoProvPlugin.createNewUser() called.");
        log.error("samlResponse.toString():
"+samlResponse.toString());
        log.error("autoProvPlugin params: "+params);
        //refer to
AbstractUserAutoProvisioning.createAndSaveNewUser();

        //parse SAMLResponse and params

        //create IEditableUser,
AbstractUserAutoProvisioning.createNPrepareNewUserForSaving()

        //create LinkIdentityCommand,
AbstractUserAutoProvisioning.prepareCmdForNewUserIdMappingLinking

        //return
AbstractUserAutoProvisioning.saveUserAndOptionallyLinkUserInOneTrx();

        return "UUID";//return new user UUID after saving to db
    }
}
```
